

Disciplina: Qualidade de Software e Testes

Professor: Leonardo Cardia

Lições Aprendidas

Gilded Rose — Modernização de Sistema Legado

Discentes:

Andre Junio Deomondes Carvalhal

Gianluca Motta Lopo

Igor Lima Carvalho

Lucca Torres Badaró Silvani

Ryan Luigi Paixão da Luz

2026

Lições Aprendidas

Reflexão sobre o processo de modernização do Gilded Rose, dificuldades enfrentadas e limitações da solução entregue.

Dificuldades

- **Capturar comportamento antes de entender a intenção:** o código legado tem regras corretas mas difíceis de extrair só de leitura (ex.: a ordem exata entre atualizar `quality` e decrementar `sell_in` importa para o resultado de "Backstage Passes" no dia do show). Foi necessário rodar o sistema (`legacy/texttest_fixture.py`) e observar a saída, não só ler o código, para confirmar a regra real antes de migrar para `src/`.
- **Approval testing mal configurado:** o teste de aprovação que já vinha no kata (`legacy/tests/test_gilded_rose_approvals.py`) falhava por falta de configuração de reporter de diff, não por bug de lógica. Isso reforçou a importância de distinguir "teste falhou porque o ambiente não está configurado" de "teste falhou porque o comportamento mudou" antes de tirar qualquer conclusão.
- **Resistir à tentação de reescrever tudo de uma vez:** era visualmente claro, desde o diagnóstico, qual seria o desenho final (Strategy por tipo de item). Ainda assim, a refatoração foi feita em passos pequenos (mover sem alterar → introduzir estratégia → adicionar Conjured), com testes rodando a cada passo, conforme o plano de contingência. Isso custou mais tempo, mas eliminou o risco de uma mudança grande mascarar uma regressão.

Aprendizados

- Testes de caracterização não substituem entender a regra de negócio — eles documentam o que o sistema faz hoje, mas a decisão de o que deveria fazer (como a regra de Conjured) ainda exige ler a especificação.
- Identificação de tipo de item por comparação de string repetida é um sintoma fácil de detectar e resolver: centralizar em constantes nomeadas e resolver a estratégia em um único ponto (`resolve_updater`) elimina a maior parte da duplicação do código original sem reescrever a regra em si.
- Separar "o que muda" (regra de cada tipo de item) de "quem orchestra" (`GildedRose`, que decide apenas chamar a estratégia certa) facilita adicionar uma nova categoria de item no futuro sem tocar nas categorias existentes — exatamente o critério de extensibilidade pedido (RNF05).

Limitações da solução entregue

- A identificação de itens Conjurados usa `name.startswith("Conjured")`. Funciona para o cenário do kata, mas é uma heurística baseada em nome — uma solução de produção real provavelmente teria um campo explícito de categoria no item, não

derivado do texto do nome. Essa limitação existe porque a classe `Item` não pode ser alterada (restrição obrigatória do trabalho).

- Cobertura de teste de 97% não inclui o script de demonstração (`src/texttest_fixture.py`), que é um script de exibição, não lógica de negócio — decisão consciente de não testar unitariamente um `print` de simulação.